

Chapter 2: Agentic AI Fundamentals

Part I: AI and Agentic Foundations

Book: Agentic AI & RAG Systems Handbook

Version: 1.0.0

Chapter ID: 02

File Name: chapter-02-agentic-ai-fundamentals.md

Chapter Overview

Generative AI systems can answer questions.

Agentic AI systems can pursue objectives.

This distinction is one of the most important concepts in modern AI architecture.

A Large Language Model (LLM) by itself is typically a reasoning engine:

1	Input
2	↓
3	Generation
4	↓
5	Output

An agent extends this model by introducing:

- Goals
- State
- Planning
- Memory
- Tools
- Observations
- Validation
- Iterative decision making

The result is a system capable of operating over multiple steps, interacting with external systems, adapting to changing conditions, and moving toward a desired outcome.

Nearly every advanced topic in this handbook, including ReAct, LangGraph, Temporal, Multi-Agent Systems, MCP, A2A, and SOP-driven operations, builds on the concepts introduced in this chapter.

Learning Objectives

After completing this chapter, you should be able to:

- Define an AI agent
 - Explain the difference between an agent and a workflow
 - Explain the difference between an agent and a microservice
 - Understand agent lifecycle management
 - Model agent state effectively
 - Understand goal-oriented execution
 - Design bounded-agent systems
 - Explain deterministic vs probabilistic execution
 - Identify appropriate use cases for agents
 - Recognize common architectural anti-patterns
-

Prerequisites

Before reading this chapter:

- Chapter 1: Generative AI Foundations
 - Basic software architecture knowledge
 - Understanding of APIs and distributed systems
-

1. Introduction

Most enterprise systems are deterministic.

Example:

```
1  if amount > 10000:
2      require_approval()
```

The software behavior is explicitly defined.

Agents behave differently.

Given a goal:

```
1  Investigate this production outage.
```

the agent may:

1	Analyze logs
2	↓
3	Query monitoring systems
4	↓
5	Retrieve historical incidents
6	↓
7	Generate hypotheses
8	↓
9	Validate evidence
10	↓
11	Recommend remediation

The exact path is not pre-programmed.

The agent determines the path dynamically.

This is what makes Agentic AI fundamentally different from traditional software systems.

2. Core Concepts

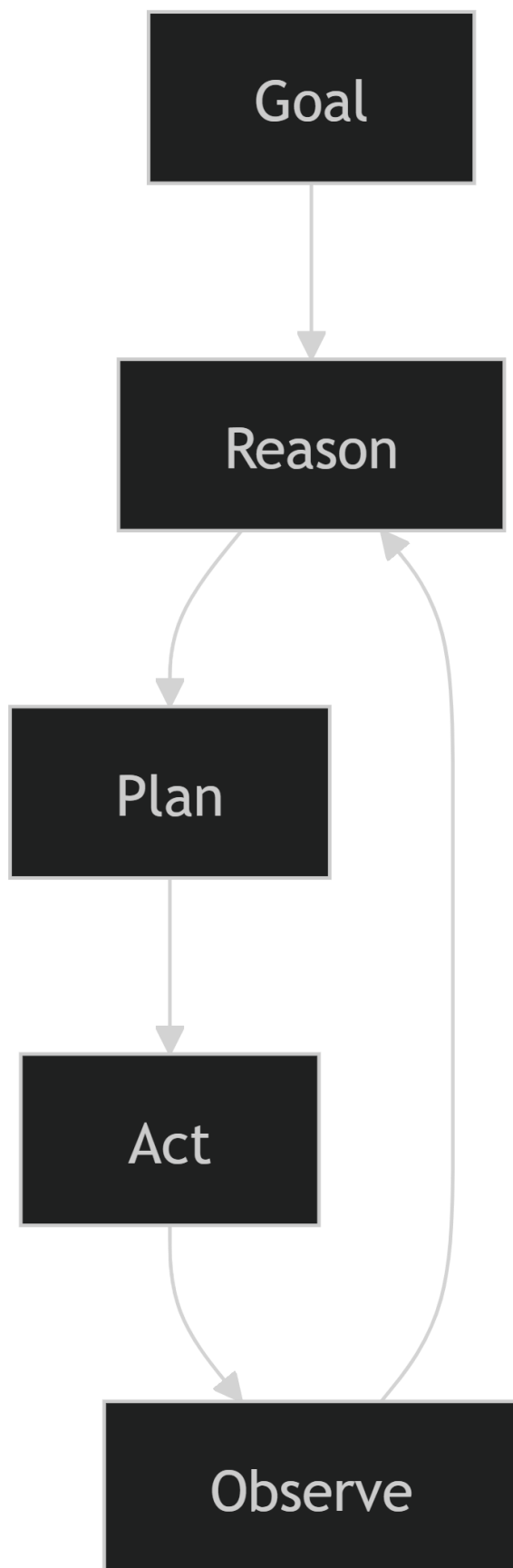
What Is an Agent?

Definition

An AI Agent is:

A goal-driven software system that can reason about a task, maintain state, use tools, make decisions, observe outcomes, and adapt its behavior to achieve an objective.

Minimal Agent Loop



The cycle continues until:

- Success
 - Failure
 - Escalation
 - Timeout
 - Human intervention
-

Essential Agent Components

Every production-grade agent requires:

Component	Purpose
Goal	Desired outcome
State	Current context
Memory	Historical information
Planner	Task decomposition
Tools	External capabilities
Observation	Feedback collection
Governance	Boundaries and controls

3. Why This Matters

Organizations increasingly require systems that can:

- Analyze incidents
- Execute workflows
- Research information
- Coordinate specialists
- Automate operations
- Support decision making

Traditional workflows struggle with uncertainty.

Agents are designed for uncertainty.

Example:

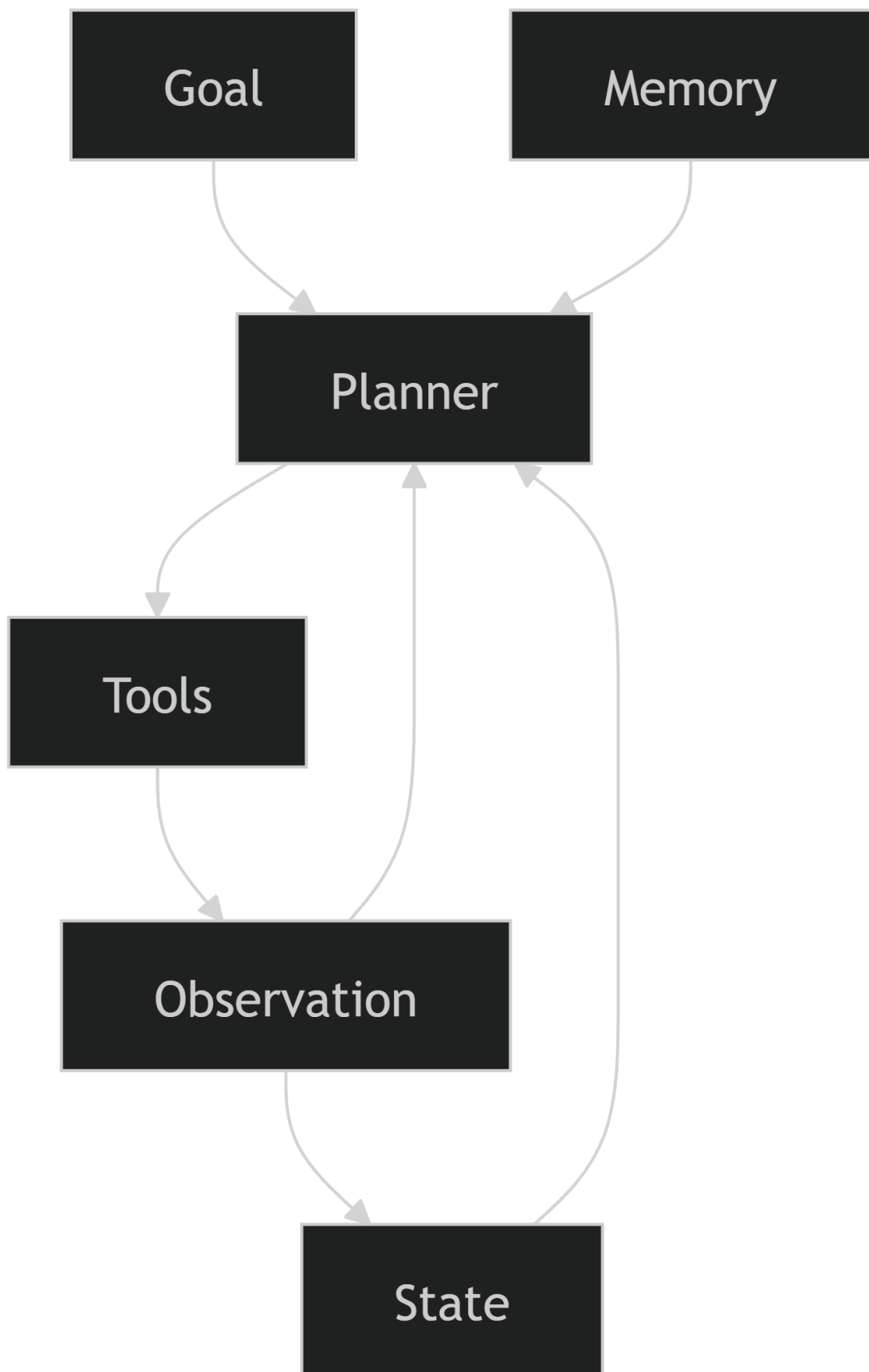
1 Find root cause of customer outage.

No deterministic workflow can anticipate every possible incident.

An agent can adapt.

4. Agent Architecture

High-Level View



The planner continuously evaluates available information and decides next actions.

5. Agent vs Workflow

One of the most common architect interview questions.

Workflow

A workflow follows predefined rules.

1	Step 1
2	↓
3	Step 2
4	↓
5	Step 3
6	↓
7	Done

Characteristics:

- Deterministic
 - Predictable
 - Auditable
 - Simple
-

Agent

An agent may dynamically select steps.

1	Goal
2	↓
3	Think
4	↓
5	Choose Action
6	↓
7	Observe
8	↓
9	Adapt

Characteristics:

- Adaptive

- Dynamic
 - Probabilistic
 - Iterative
-

Comparison

Feature	Workflow	Agent
Steps	Fixed	Dynamic
Planning	None	Required
Adaptation	Minimal	High
Predictability	High	Lower
Governance Complexity	Lower	Higher
Cost	Lower	Higher

Architecture Guidance

Use workflows when:

- Process is known
- Path is predictable
- Audit requirements are strict

Use agents when:

- Uncertainty exists
 - Multiple paths are possible
 - Dynamic reasoning adds value
-

6. Agent vs Microservice

Another frequent Staff/Principal interview topic.

Microservice

A microservice performs deterministic business functionality.

Example:

1 Invoice Service

Input:

```
1 {  
2   "invoice_id": "123"  
3 }
```

Output:

```
1 Invoice Details
```

Behavior is fixed.

Agent

Input:

```
1 Analyze invoice issues.
```

Output may vary depending on:

- Context
- Policies
- Data
- History
- Retrieved information

Reasoning determines execution.

Comparison

Dimension	Microservice	Agent
Logic	Deterministic	Probabilistic
Inputs	Structured	Structured + Unstructured
Decisions	Explicit	Dynamic
State	Optional	Critical
Tool Usage	Not Typical	Common
Planning	None	Required

Architect Perspective

Best enterprise design:

```
1 Agents  
2 Reason  
3  
4 Microservices  
5 Execute
```

Important Principle:

Agents decide. Deterministic systems execute.

7. Deterministic vs Probabilistic Systems

Deterministic

Given identical inputs:

1	Same Input
2	↓
3	Same Output

Examples:

- Authentication
 - Authorization
 - Billing
 - SQL queries
 - Financial calculations
-

Probabilistic

Output may vary.

Examples:

- Summarization
 - Classification
 - Root cause analysis
 - Content creation
 - Planning
-

Enterprise Architecture Rule

1	Probabilistic Layer
2	↓
3	Deterministic Validation
4	↓
5	Execution Layer

Never allow:

1	LLM
2	↓
3	Production Action

without validation.

8. Agent Lifecycle

Lifecycle Overview



Stage 1: Goal Acquisition

Examples:

1	Find root cause.
2	
3	Generate compliance report.
4	
5	Investigate anomaly.

A poor goal definition leads to poor agent performance.

Stage 2: Context Collection

Sources:

- User input
 - Memory
 - Knowledge base
 - Documents
 - APIs
-

Stage 3: Planning

Agent determines:

1 What should I do next?

Example:

1 Retrieve logs
2 ↓
3 Analyze
4 ↓
5 Validate

Stage 4: Execution

Tools perform actions.

Examples:

- Search
 - Database access
 - Ticket creation
 - API calls
-

Stage 5: Observation

Gather results.

Examples:

1 Tool output
2 System response
3 Errors
4 Warnings

Stage 6: Replanning

Agent decides:

1 Continue?
2 Retry?
3 Escalate?
4 Complete?

Stage 7: Completion

Produce result.

Example:

```
1 Root Cause Found
```

9. Agent State

Definition

State represents everything the agent currently knows about an execution.

Example State

```
1 from typing import TypedDict
2
3 class AgentState(TypedDict):
4     objective: str
5     evidence: list
6     findings: list
7     next_step: str
8     completed: bool
```

Why State Matters

Without state:

```
1 Agent Forgets
```

With state:

```
1 Agent Progresses
```

State Categories

Type	Description
Business State	Domain information
Workflow State	Execution progress
Agent State	Reasoning context
System State	Runtime metadata

10. Agent Memory

Agent memory differs from state.

State

Current execution.

Memory

Persistent history.

Example

State:

1	Current investigation
---	-----------------------

Memory:

1	Past investigations
---	---------------------

Memory Types

Working Memory

Active execution.

Short-Term Memory

Current session.

Episodic Memory

Experiences.

Semantic Memory

Facts.

Procedural Memory

Processes.

Long-Term Memory

Persistent knowledge.

11. Types of Agents

Assistant Agents

Help users.

Examples:

- Copilots
 - Search assistants
-

Task Agents

Perform focused tasks.

Examples:

- SQL generation
 - Research
 - Classification
-

Workflow Agents

Operate within business processes.

Examples:

- Ticket routing
 - Approval workflows
-

Autonomous Agents

High independence.

Examples:

- Research systems
 - Monitoring systems
 - Investigation systems
-

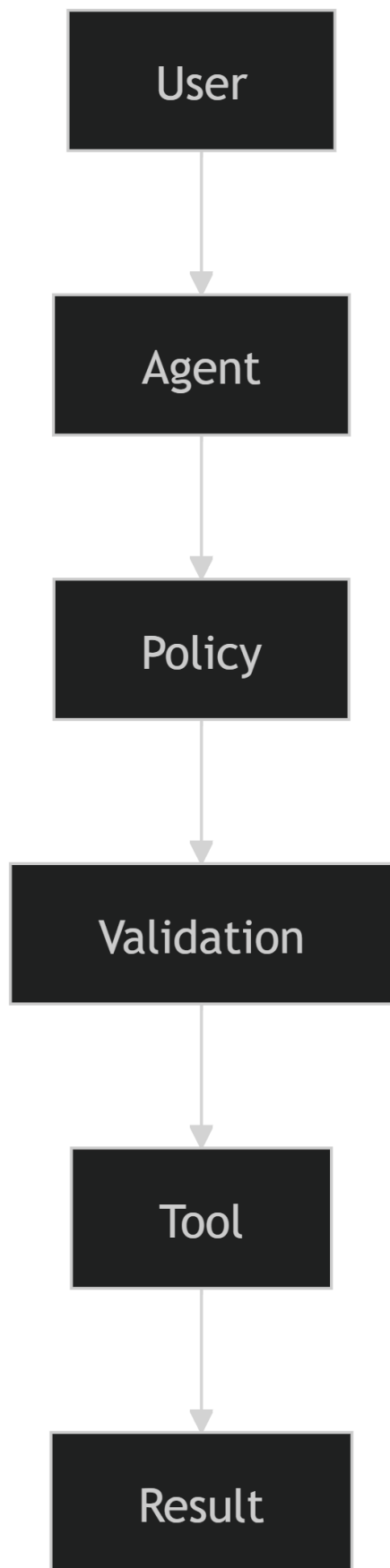
12. Agent Governance

Agentic systems require governance.

Without governance:

- | | |
|---|-----------|
| 1 | Reasoning |
| 2 | = |
| 3 | Risk |

Governance Layer

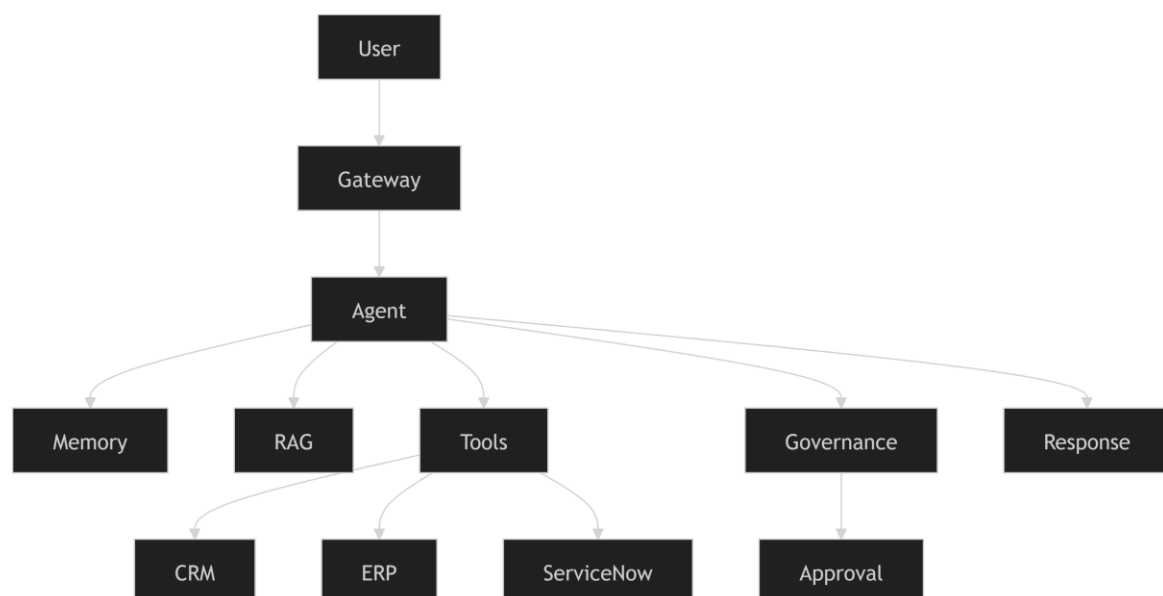


Governance Questions

Every agent must answer:

- Who can use me?
 - What can I access?
 - Which tools may I invoke?
 - What actions require human approval?
 - What should be audited?
-

13. Enterprise Agent Architecture



14. Production Example

IT Incident Agent

Goal:

1 Investigate production failure.

Workflow:

1 Logs
2 ↓

3	Metrics
4	↓
5	Past Incidents
6	↓
7	Root Cause
8	↓
9	Recommendation

Tools:

- Splunk
- Datadog
- ServiceNow
- Knowledge Repository

Governance:

- Read access only
 - Human approval for remediation
-

15. Security

Agent security starts with identity.

Controls include:

- Authentication
- Authorization
- Tool permissions
- Audit logging
- Prompt validation
- Retrieval controls

Never grant:

1	Every Agent
2	↓
3	Every Tool

16. Observability

Track:

- Agent success rate
 - Steps per execution
 - Tool usage
 - Retries
 - Cost
 - Latency
 - Approval requests
-

17. Common Mistakes

Mistake 1

Using agents for deterministic workflows.

Mistake 2

No state model.

Mistake 3

No governance layer.

Mistake 4

Unlimited tool access.

Mistake 5

No memory strategy.

Mistake 6

No observability.

Mistake 7

Confusing workflows and agents.

18. Anti-Patterns

Agent God Mode

- | | |
|---|------------|
| 1 | One Agent |
| 2 | Everything |

3 Every Tool

Fails at scale.

Agent For Everything

Not all problems require agents.

Sometimes a workflow is better.

Hidden State

State changes invisible to operators.

Difficult to debug.

Unlimited Autonomy

High operational risk.

19. Architecture Decision Matrix

Requirement	Agent	Workflow
Fixed Process	Poor	Excellent
Dynamic Reasoning	Excellent	Poor
Uncertainty	Excellent	Moderate
Governance Simplicity	Lower	Higher
Auditability	Moderate	Strong
Adaptability	Strong	Limited

20. Real-World Use Cases

Compliance Advisor

Policy interpretation.

Incident Investigator

Root cause analysis.

Customer Support Copilot

Case resolution.

Knowledge Assistant

Enterprise search.

Research Agent

Information discovery.

21. Interview Questions

Q1

What is an AI agent?

Q2

What differentiates an agent from an LLM?

Q3

What differentiates an agent from a workflow?

Q4

What differentiates an agent from a microservice?

Q5

Why is state important?

Q6

Why is memory important?

Q7

What is the biggest enterprise risk when implementing agents?

Q8

When should you NOT use an agent?

22. Interview Answers

Q1

A goal-oriented system capable of reasoning, planning, tool usage, observation, memory utilization, and iterative execution.

Q2

An LLM generates responses. An agent uses reasoning plus planning, state, memory, and tools to achieve objectives.

Q3

Workflows follow predefined paths; agents determine paths dynamically.

Q4

Microservices execute deterministic functionality. Agents perform probabilistic reasoning.

Q5

State enables multi-step execution.

Q6

Memory enables continuity and learning.

Q7

Uncontrolled autonomy combined with insufficient governance.

Q8

When workflow behavior is fully deterministic and known.

23. Architecture Review Checklist

Agent Fundamentals

- Goal clearly defined
- State model documented
- Memory model documented
- Tool permissions defined
- Governance layer present
- Human approval requirements defined
- Observability implemented

- Security controls documented
 - Failure handling documented
-

24. Practical Exercises

1. Design an Incident Investigation Agent.
 2. Compare agents and workflows for customer onboarding.
 3. Define agent state for a compliance review process.
 4. Draw an enterprise agent architecture.
 5. Design governance controls for a deployment agent.
-

25. Key Takeaways

- Agents are goal-driven systems that reason and act.
 - Agentic systems extend LLMs with planning, state, memory, and tools.
 - Agents are not workflows.
 - Agents are not microservices.
 - State and memory serve different purposes.
 - Governance is mandatory.
 - Deterministic systems should remain execution authorities.
 - Enterprise agents require security, observability, and approval controls.
 - Not every problem should be solved with an agent.
 - Strong agent architecture balances autonomy with governance.
-

26. References

- ReAct: Synergizing Reasoning and Acting in Language Models
 - LangGraph Documentation
 - Temporal Documentation
 - OpenAI Platform Documentation
 - Anthropic Agent Design Guidance
 - Microsoft Agent Framework Documentation
 - AWS Agentic AI Reference Architectures
-

Chapter Status: Completed

Markdown Ready: Yes

PDF Ready: Yes (pending build pipeline)

Next Chapter: Chapter 3 – Agent Design Principles and Lifecycle Engineering